NODE.JS BACKEND INTERVIEW PREP

Event Loop & Async Output Questions

A practical workbook with one priority method, 11 interview-style code questions, step-by-step dry runs, tricky cases, and a final-page answer key.

WHAT YOU WILL MASTER

Synchronous execution, Promise microtasks, process.nextTick(), async/await, timers, setImmediate(), file I/O, nested callbacks, and CommonJS vs ES modules.

HOW TO USE THIS GUIDE

First learn the checklist. Then predict each output before reading the dry run. On the final page, revise all 11 questions from their queue traces and output summaries.

Grounded in current official Node.js documentation. Timing-dependent cases are labelled rather than assigned a false fixed output.

1. The final priority method

Use this as a solving checklist, not as a universal global ranking for every callback. The execution context matters.

Step	What to inspect	How to reason
1. Synchronous code	Ordinary statements, function calls, code before await	Run in source order until the current call stack completes.
2. Registration	setTimeout, setImmediate, I/O, Promise.then	Registering work is not a separate phase and does not mean its callback runs immediately.
3. process.nextTick()	The next-tick queue	In typical CommonJS top-level execution, queued callbacks run before Promise microtasks.
4. Promise microtasks	then/catch/finally, await continuations, queueMicrotask	Preserve queue order. A callback added during a drain goes behind already queued microtasks.
5. Event-loop callbacks	Timers, I/O/poll, setImmediate/check, close callbacks	Follow the callback context and phase; do not force all callbacks into one fixed priority list.

The five-step dry-run routine

1. Mark synchronous output and run all direct function calls in order.
2. Write the initial queues: nextTick, Promise microtasks, timers, I/O callbacks and immediates.
3. Drain the applicable queues, tracking callbacks added while a queue is being processed.
4. Identify context: CommonJS or ESM? Top-level or inside I/O/timer callback?
5. Apply phases and uncertainty: state the guaranteed prefix and identify any timing-dependent ending.

> Critical correction: “Callbacks are registered” is a step in reading the source, not an execution-priority slot. And “nextTick always runs before Promise” is too broad: CommonJS and ES module contexts can differ.

2. Event-loop phases and special cases

The following is the conceptual phase order from the Node.js event-loop guide. The loop repeats these phases; not every phase runs a callback on every pass.

Phase	Typical callback / role
1. Timers	Eligible setTimeout() and setInterval() callbacks. A timer delay is a threshold, not an exact execution time.
2. Pending callbacks	Certain deferred system-level callbacks, such as some TCP error callbacks.
3. Idle / Prepare	Internal Node.js and libuv operations.
4. Poll	Wait for and process available I/O callbacks; file/network I/O callbacks are commonly associated with this phase.
5. Check	setImmediate() callbacks.
6. Close callbacks	Certain close events, for example socket close callbacks.

Rules interviewers often test

Case	Reliable rule
Top-level setTimeout(fn, 0) vs setImmediate()	Either can run first. Do not claim a universal winner.
Inside an I/O callback	setImmediate() generally runs before a newly scheduled setTimeout(fn, 0).
setTimeout(fn, 5000) vs setImmediate()	Under normal execution, immediate is expected much earlier; the timer cannot be eligible before its delay has elapsed.
Promise schedules process.nextTick()	The new nextTick does not interrupt a Promise microtask queue that is currently being drained.
CommonJS vs ES modules	Initial nextTick/Promise ordering can differ because top-level ESM evaluation is already part of microtask processing.
Recursive process.nextTick()	Can starve I/O if it continually replenishes the next-tick queue.

Version note: Since Node.js 20 (libuv 1.45), timer processing changed to run after poll in each loop iteration, with a compatibility timer pass before entering the loop. This can affect timing-sensitive timer/I/O cases.

Question 1 of 11 | Easy

Regular functions and call-stack order

This pattern checks whether you confuse LIFO stack behavior with ordinary sequential function calls.

Code

```
function first() {
  console.log("first");
}

function second() {
  console.log("second");
}

function third() {
  console.log("third");
}

first();
second();
third();
```

Step-by-step dry run

Step 1: The function declarations are evaluated; they do not print anything.

Step 2: Call first(); it prints first and returns.

Step 3: Call second(); it prints second and returns.

Step 4: Call third(); it prints third and returns.

Result: first -> second -> third

Why this result? The three calls are separate, sequential top-level statements. LIFO describes nested stack frames: if first() called second(), the inner call would finish before first() resumed.

Queue / context note: No asynchronous queues or event-loop phases are involved.

Question 2 of 11 | Easy

Synchronous code vs Promise vs timer

This is the essential baseline for nearly every output-based event-loop question.

Code

```
console.log("A");

setTimeout(() => console.log("B"), 0);

Promise.resolve().then(() => console.log("C"));

console.log("D");
```

Step-by-step dry run

Step 1 - Synchronous: print A.

Step 2 - Register: schedule the timer for B and queue the Promise callback for C.

Step 3 - Synchronous: print D; the current script completes.

Step 4 - Microtasks: print C.

Step 5 - Timer: when eligible, print B.

Result: A -> D -> C -> B

Why this result? The synchronous script finishes before Promise microtasks. The Promise callback is processed before the eligible timer callback in this ordinary example.

Queue / context note: Before async callbacks: Promise queue [C]; timer queue includes B.

Question 3 of 11 | Easy-Medium

nextTick vs Promise vs timer vs immediate

This is the mixed top-level pattern from your screenshot. The last two callbacks have no guaranteed relative order.

Code

```
console.log("start");

setTimeout(() => console.log("timeout"), 0);
setImmediate(() => console.log("immediate"));
Promise.resolve().then(() => console.log("promise"));
process.nextTick(() => console.log("tick"));

console.log("end");
```

Step-by-step dry run

Step 1 - Synchronous: print start, then end.

Step 2 - CommonJS nextTick: print tick.

Step 3 - Promise microtask: print promise.

Step 4 - Event loop: timeout and immediate both remain; their top-level order is not guaranteed.

> Result: Guaranteed prefix: start -> end -> tick -> promise. Ending: timeout -> immediate OR immediate -> timeout.

Why this result? A top-level setTimeout(fn, 0) and setImmediate() can run in either order. Never force a single ending for this code.

Queue / context note: This dry run assumes CommonJS (.cjs or CommonJS .js), not ES module top-level evaluation.

Question 4 of 11 | Medium

Nested Promise callbacks

This question tests FIFO order inside the Promise microtask queue.

Code

```
console.log("A");

Promise.resolve().then(() => {
  console.log("B");
  Promise.resolve().then(() => console.log("C"));
});

Promise.resolve().then(() => console.log("D"));

console.log("E");
```

Step-by-step dry run

Step 1 - Synchronous: print A; register callbacks B and D; print E. Output so far: A, E.

Step 2 - Initial Promise queue: [B callback, D callback].

Step 3: B runs and schedules C at the back, so the queue becomes [D callback, C callback].

Step 4: D runs, then C runs.

Result: A -> E -> B -> D -> C

Why this result? D was already waiting when B queued C. New Promise microtasks join the back of the currently pending queue.

Queue / context note: Queue trace: [B, D] -> [D, C] -> [C] -> [].

Question 5 of 11 | Medium

nextTick scheduled inside a Promise

This is the nested queue example we dry-ran together.

Code

```
console.log("Start");

Promise.resolve().then(() => {
  console.log("P1");
  process.nextTick(() => console.log("N2"));
});

process.nextTick(() => console.log("N1"));
Promise.resolve().then(() => console.log("P2"));

console.log("End");
```

Step-by-step dry run

Step 1 - Synchronous: print Start, register P1, register N1, register P2, then print End.

Step 2 - Initial CommonJS nextTick queue: N1 runs.

Step 3 - Promise queue: P1 runs and queues N2; P2 was already waiting, so P2 runs next.

Step 4: after the active Promise microtask drain, the newly queued N2 runs.

Result: Start -> End -> N1 -> P1 -> P2 -> N2

Why this result? The nextTick callback created inside P1 does not interrupt the Promise microtask drain that is already in progress.

Queue / context note: Initial: nextTick [N1], Promise [P1, P2]. After P1: nextTick [N2], Promise [P2].

Question 6 of 11 | Medium-Hard

Top-level timer vs file I/O vs immediate

This mirrors your screenshot that combines setTimeout, fs.readFile and setImmediate.

Code

```
const fs = require("node:fs");

setTimeout(() => console.log("timeout"), 0);

fs.readFile(__filename, () => {
  console.log("file");
});

setImmediate(() => console.log("immediate"));
```

Step-by-step dry run

Step 1 - Synchronous setup: register the timer, initiate file read, register the immediate callback.

Step 2: the timer and immediate become eligible in their respective timing/phase contexts.

Step 3: the file callback runs when the read completes and Node processes the I/O event.

Step 4: do not assume a fixed total ordering among all three callbacks.

> Result: No single total order is guaranteed. The timer/immediate order can vary; the file callback depends on I/O completion/readiness.

Why this result? The top-level timer/immediate relation is nondeterministic, and file-read completion is asynchronous. A single observed run is not a universal rule.

Queue / context note: Use a real file in the script directory when running this example. __filename is CommonJS-specific.

Question 7 of 11 | Medium-Hard

setImmediate vs timer inside an I/O callback

This is the key contrast with Question 6.

Code

```
const fs = require("node:fs");

fs.readFile(__filename, () => {
  console.log("file");
  setTimeout(() => console.log("timeout"), 0);
  setImmediate(() => console.log("immediate"));
});
```

Step-by-step dry run

Step 1: the file read completes and the I/O callback starts.

Step 2: print file; schedule both timeout and immediate from inside the I/O callback.

Step 3: immediate is processed in the Check phase after poll processing.

Step 4: the zero-delay timer runs when eligible in later timer processing.

Result: file -> immediate -> timeout (expected ordering)

Why this result? When both are scheduled inside an I/O callback, setImmediate() generally executes before the newly scheduled setTimeout(fn, 0).

Queue / context note: The I/O callback context is what makes this differ from a top-level example.

Question 8 of 11 | Hard

Full mixed example: file, timer, Promise, nextTick, immediate

This combines all of the major top-level patterns shown in your screenshots.

Code

```
const fs = require("node:fs");

console.log("start");
setTimeout(() => console.log("timeout"), 0);
fs.readFile(__filename, () => console.log("file"));
Promise.resolve().then(() => console.log("promise"));
process.nextTick(() => console.log("tick"));
setImmediate(() => console.log("immediate"));
console.log("end");
```

Step-by-step dry run

Step 1 - Synchronous: print start, register async work, then print end.

Step 2 - CommonJS nextTick: tick.

Step 3 - Promise microtask: promise.

Step 4 - Event loop: file, timeout and immediate are processed according to I/O readiness and phase timing; their complete relative order is not universally fixed.

> Result: Guaranteed prefix: start -> end -> tick -> promise. Remaining callbacks: file, timeout and immediate; do not assert one universal total order.

Why this result? The synchronous and initial microtask prefix is deterministic under CommonJS. The remaining callbacks include I/O and top-level timer/immediate scheduling, where timing matters.

Queue / context note: A good interview answer states the guaranteed prefix, then clearly labels the variable suffix.

Question 9 of 11 | Medium-Hard

async/await vs Promise microtasks

An async function starts executing immediately, then pauses at await.

Code

```
async function demo() {
  console.log("A");
  await Promise.resolve();
  console.log("B");
}

console.log("C");
demo();
console.log("D");
Promise.resolve().then(() => console.log("E"));
```

Step-by-step dry run

Step 1 - Synchronous: print C; call demo(), which immediately prints A.

Step 2: demo pauses at await; print D from the outer script and queue E.

Step 3 - Microtasks: the continuation after await was queued before the later .then callback, so B prints before E.

Result: C -> A -> D -> B -> E

Why this result? The code before await is synchronous. The continuation after await is asynchronous and is queued before the later .then callback in this example.

Queue / context note: No timers or I/O are needed for this output.

Question 10 of 11 | Advanced

process.nextTick vs Promise vs queueMicrotask

This question exposes a real CommonJS vs ES module difference.

Code

```
console.log("sync");
process.nextTick(() => console.log("nextTick"));
Promise.resolve().then(() => console.log("promise"));
queueMicrotask(() => console.log("microtask"));
```

Step-by-step dry run

Step 1: both files print sync first.

Step 2 - CommonJS (.cjs): nextTick queue drains before the microtask queue.

Step 3 - CommonJS: Promise callback was queued before queueMicrotask, so promise then microtask.

Step 4 - ES module (.mjs): top-level module evaluation is already in the microtask context, so promise and microtask run before nextTick.

Result: CommonJS: sync -> nextTick -> promise -> microtask.
ES module: sync -> promise -> microtask -> nextTick.

Why this result? Node.js documents different initial ordering because ES module evaluation itself happens as part of microtask processing.

Queue / context note: Run separately as node example.cjs and node example.mjs to compare.

Question 11 of 11 | Advanced

Five-second timer vs immediate with queues

A longer timer is not the same problem as setTimeout(fn, 0).

Code

```
console.log("Start");

setTimeout(() => console.log("Timeout 5 seconds"), 5000);
setImmediate(() => console.log("Immediate"));
Promise.resolve().then(() => console.log("Promise"));
process.nextTick(() => console.log("Tick"));

console.log("End");
```

Step-by-step dry run

Step 1 - Synchronous: Start, then End.

Step 2 - CommonJS nextTick: Tick.

Step 3 - Promise: Promise.

Step 4 - Event loop: Immediate is eligible during the Check phase; the timer cannot be eligible until its approximately five-second threshold has elapsed.

Result: Start -> End -> Tick -> Promise -> Immediate -> Timeout 5 seconds (normal execution)

Why this result? A timer delay is a minimum threshold, not exact scheduling. Under normal execution, a setImmediate callback occurs long before a five-second timer is eligible.

Queue / context note: The process remains alive for the referenced five-second timer.

3. Reference notes and how to use this guide

These questions are a curated study set covering core interview patterns, not a statistically ranked list of the globally most frequent interview questions. Answers are based on Node.js documented behavior; timing-dependent cases are intentionally labelled.

Official Node.js documentation

The Node.js Event Loop - phases, poll, timers, setImmediate and setTimeout
https://nodejs.org/learn/asynchronous-work/event-loop-timers-and-nexttick

process.nextTick() and queueMicrotask() - current process API docs
https://nodejs.org/api/process.html#when-to-use-queuemicrotask-vs-processnexttick

Timers API - setTimeout and setImmediate
https://nodejs.org/api/timers.html

ECMAScript modules (ESM) - module behavior and CommonJS differences
https://nodejs.org/api/esm.html

File system API - readFile callback behavior
https://nodejs.org/api/fs.html#fsreadfilepath-options-callback

Current API note: Current Node.js documentation labels process.nextTick() as Legacy and recommends queueMicrotask() for most userland deferral needs. However, process.nextTick() remains important to understand for interviews and for reading existing Node.js code.

Suggested practice routine

For each question, first solve it on paper. Write the synchronous output, make explicit queue lists, and only then process callbacks. Run timing-dependent questions multiple times on your own Node.js version and record the guaranteed behavior separately from one observed run.

Before interviews, review the final page until you can explain every output and state why any ordering is not guaranteed.

Final page: all 11 answers + dry-run traces

Fast revision. Steps are deliberately compressed; use the full question pages for the detailed explanation. CJS = CommonJS; ESM = ES module.

Q1. Function calls
Dry run: Declarations print nothing; call first(), then second(), then third().
Answer: first -> second -> third

Q2. Sync + Promise + timer
Dry run: Sync A,D; drain Promise C; then eligible timer B.
Answer: A -> D -> C -> B

Q3. Tick + Promise + timer + immediate
Dry run: Sync start,end; CJS nextTick tick; Promise promise; timer/immediate may swap.
Answer: Guaranteed: start -> end -> tick -> promise; then timeout/immediate in either order.

Q4. Nested Promise
Dry run: Sync A,E; queue [B,D]; B adds C behind D; then D, then C.
Answer: A -> E -> B -> D -> C

Q5. nextTick inside Promise
Dry run: Sync Start,End; N1; Promise P1 adds N2; finish P2 before N2.
Answer: Start -> End -> N1 -> P1 -> P2 -> N2

Q6. Top-level file I/O + timer + immediate
Dry run: Register timer, start read, register immediate; file fires on I/O completion; no fixed total order.
Answer: No single guaranteed total order.

Q7. Immediate inside I/O
Dry run: File callback prints file; its setImmediate runs in Check before its newly scheduled zero timer is eligible.
Answer: file -> immediate -> timeout (expected)

Q8. Mixed screenshot pattern
Dry run: Sync start,end; CJS nextTick tick; Promise promise; file/timer/immediate order depends on I/O and scheduling.
Answer: Guaranteed prefix: start -> end -> tick -> promise; suffix is timing-dependent.

Q9. async/await
Dry run: Sync C; demo prints A then pauses; outer prints D; await continuation B was queued before E.
Answer: C -> A -> D -> B -> E

Q10. nextTick + Promise + queueMicrotask
Dry run: CJS: drain nextTick, then Promise and queueMicrotask in queue order. ESM: Promise/microtask before nextTick.
Answer: CJS: sync -> nextTick -> promise -> microtask; ESM: sync -> promise -> microtask -> nextTick.

Q11. 5-second timer + immediate
Dry run: Sync Start,End; CJS Tick; Promise; Immediate in Check well before 5-second timer becomes eligible.
Answer: Start -> End -> Tick -> Promise -> Immediate -> Timeout 5 seconds

Master rule: Synchronous execution -> applicable nextTick queue -> Promise microtasks -> relevant event-loop phase. Re-check queues after callbacks; distinguish guaranteed order from timing-dependent order.